ROS 2 Executor

Based on **The Evolution of Execution Management in rclcpp**
by *Skye Medeiros* (Polymath Robotics), May 2026.

fujitatomoya@github / tomoyafujita@linkedin

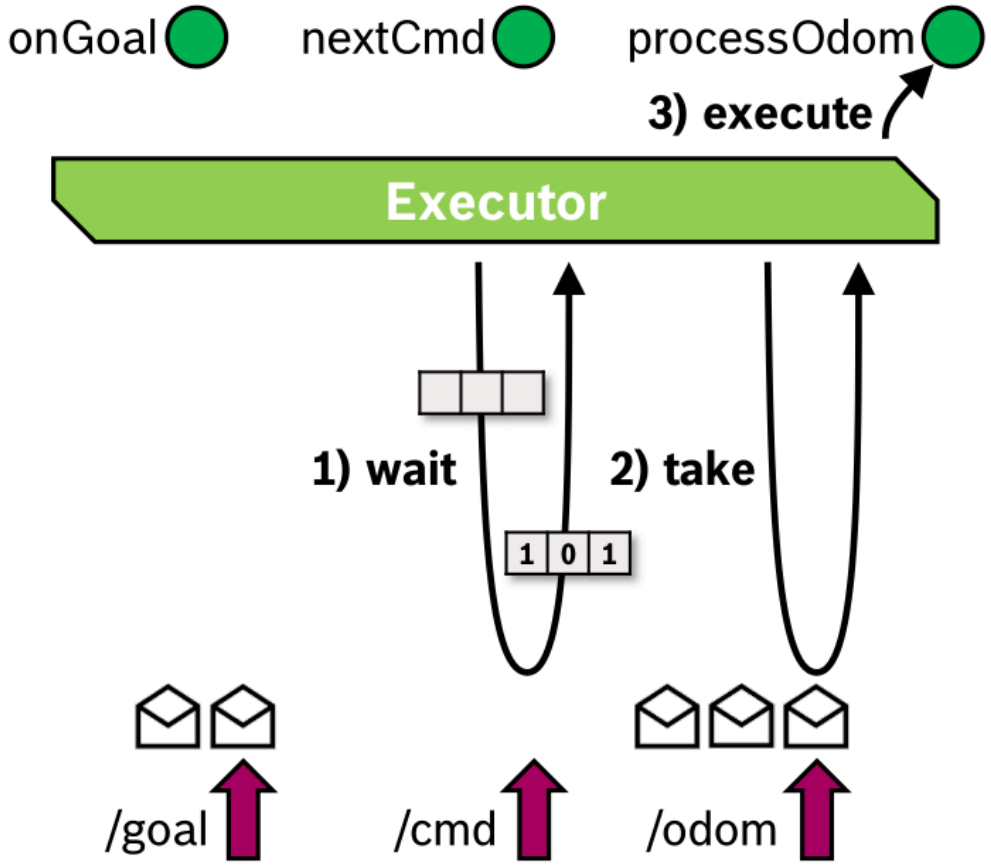
Agenda

- What an Executor is, and the moving parts
- The classic execution model
  - Single-threaded / Multi-threaded / Callback groups
- Where it breaks down
  - Polling overhead, serialized discovery
- The paradigm shift: `EventsExecutor`
- `EventsCBGExecutor` — closing the gaps
- Benchmarks, caveats, and what to use



The Moving Parts, In Brief

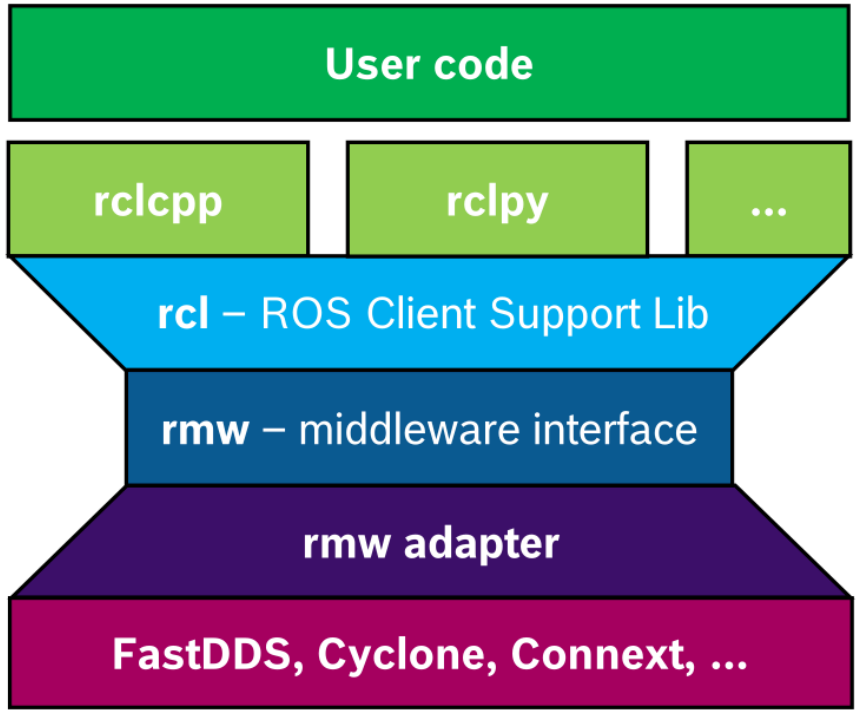
- Your node owns **entities**: subscriptions, service / action servers & clients, timers, waitables.
- Entities do **not** run themselves — something must notice they are ready and invoke the callback.
- That "something" is the **Executor**, living in the client library (`rclcpp` / `rclpy`), above `rcl` and `rmw` .



Three Important Concepts

- **Executor** — owns the thread(s) that invoke callbacks. `spin()` is its main loop.
- **Callback Group** — the developer's declaration of *what is safe to run in parallel*.
- **Wait Set** — a flat collection of entities handed down to the middleware to wait on, rebuilt every iteration.

The wait set is the heart of the *classic* model — and the thing the new model deletes.

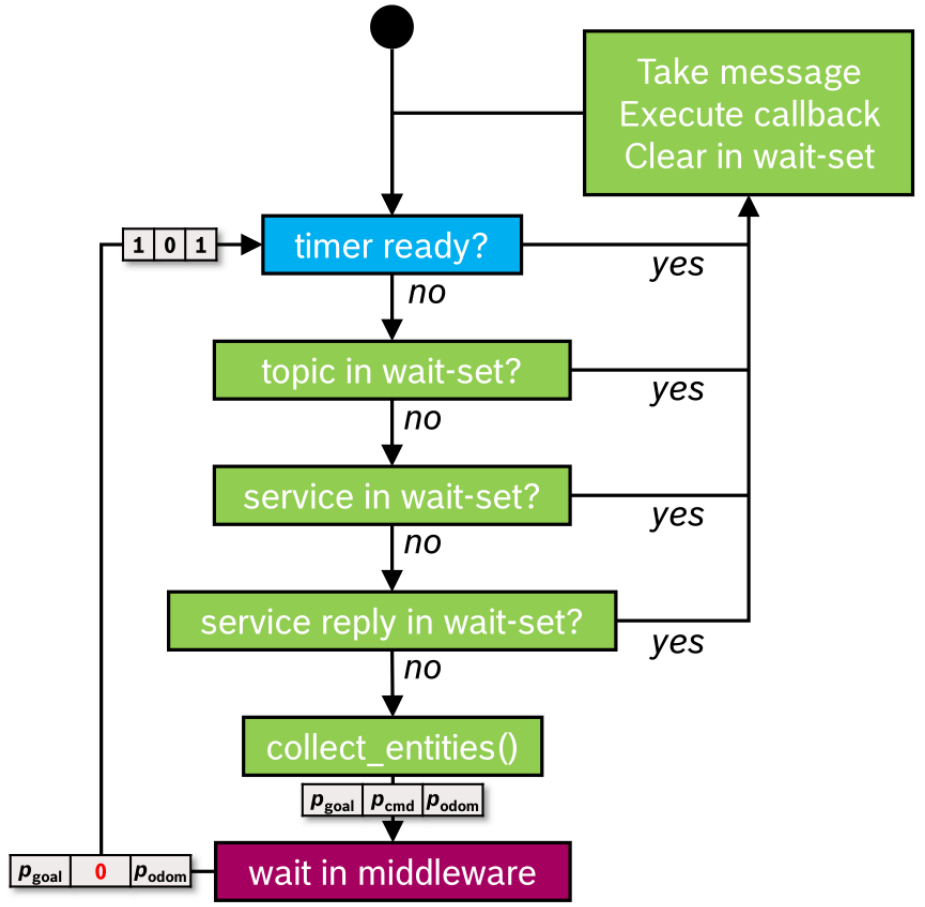


The Classic Execution Model

The loop is deliberately simple:

1. Build the **wait set** from all entities.
2. Ask the middleware: *who is ready?*
3. Pick one ready entity → run its callback.
4. Repeat.

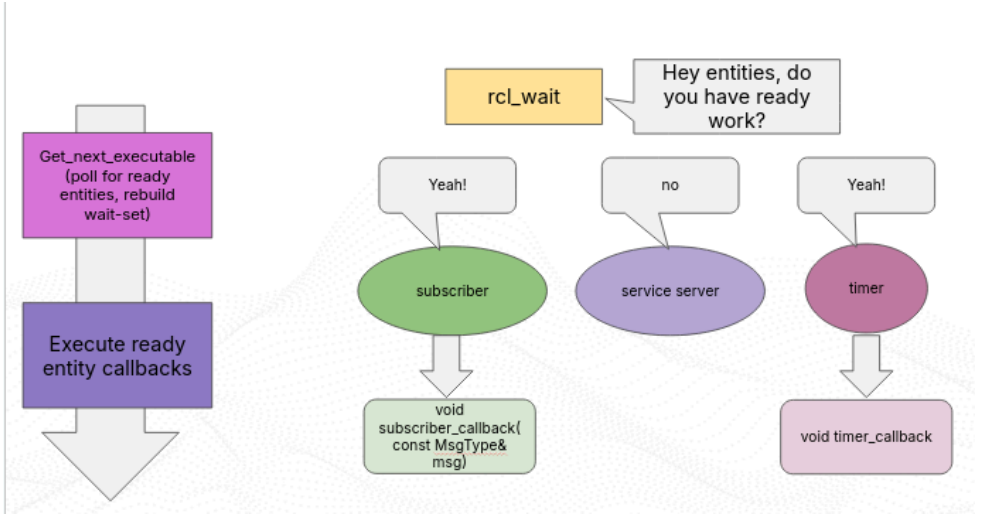
Two halves: **discover work**, then **do work**.



`SingleThreadedExecutor`

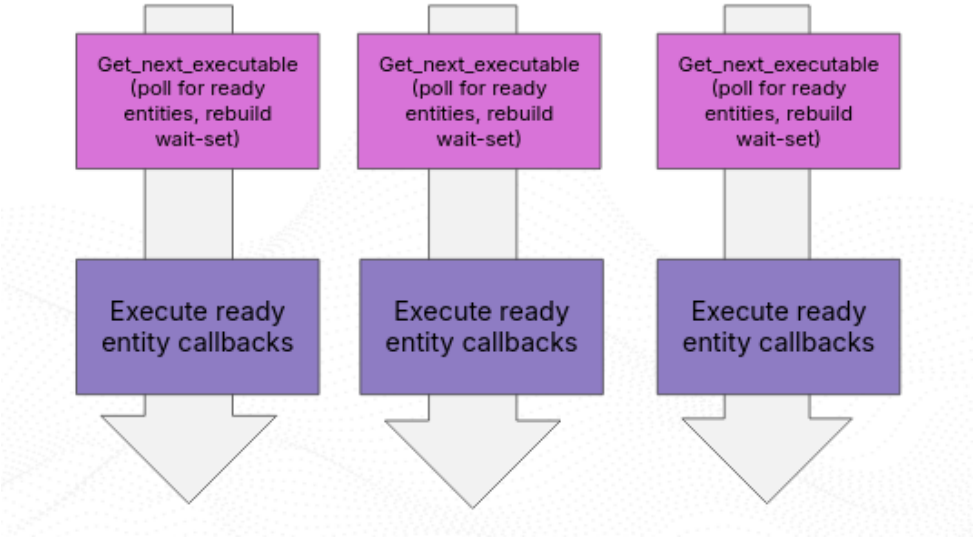
- What `rclcpp::spin(node)` gives you by default.
- **One thread**, one callback at a time, strictly sequential.
- Multiple nodes can share it — that is exactly what **component containers** do.

Simple, predictable, no data races between callbacks — and no parallelism at all.



`MultiThreadedExecutor` + Callback Groups

- Several **worker threads** pull ready callbacks from a shared pool.
- Concurrency is bounded by **callback groups**, so developers stay in control.



Callback Groups — the Concurrency Contract

Type	Guarantee
Mutually Exclusive	At most **one** callback in the group runs at a time
Reentrant	Callbacks in the group may run **in parallel** with each other
Separate groups	Always free to run in parallel with one another

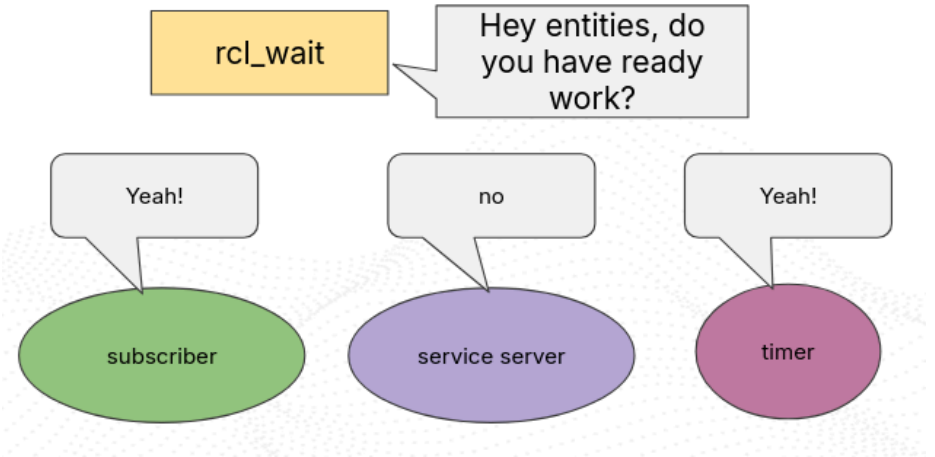
Default: every entity lands in the node's default **mutually exclusive** group.

Polling Overhead

The executor asks **every** entity whether it is ready, on **every** iteration.

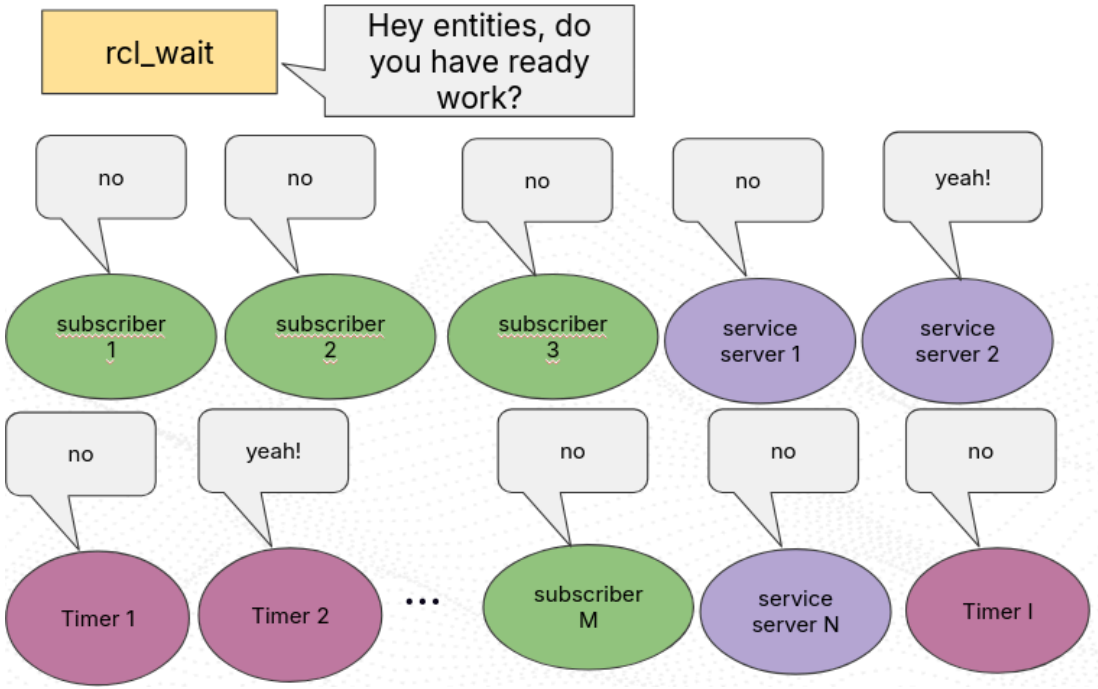
"The runtime cost of checking which entities are ready **scales linearly with the number of entities**."

One busy subscription among 49 idle ones costs the same to discover as 50 busy ones.



It Gets Worse With Scale

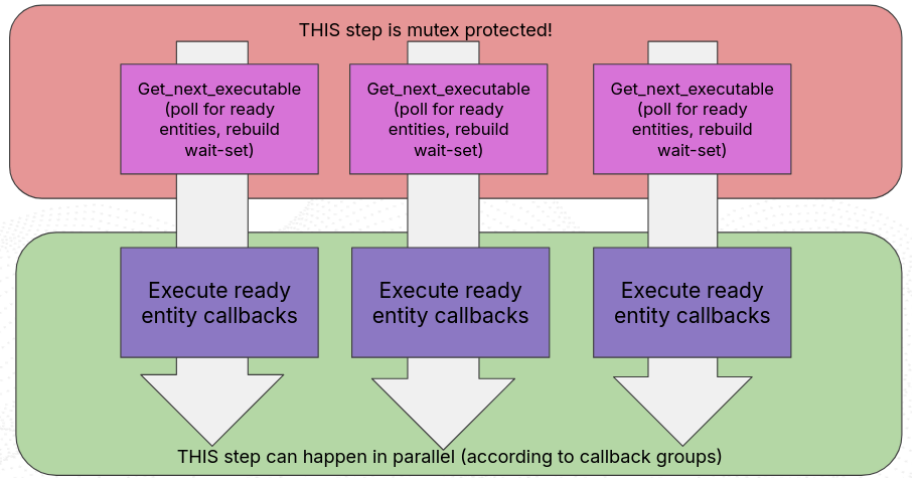
- Discovery cost grows with entity **count**, not with actual **traffic**.
- Real systems (Nav2, `rosbag2`, perception pipelines) have *many* entities per process.
- Especially painful on constrained hardware — ARM SBCs, where "the bottlenecks and resulting CPU contention become so much more apparent."



How Parallel, Actually?

Every worker thread must take `wait_mutex_` before it can even look for work:

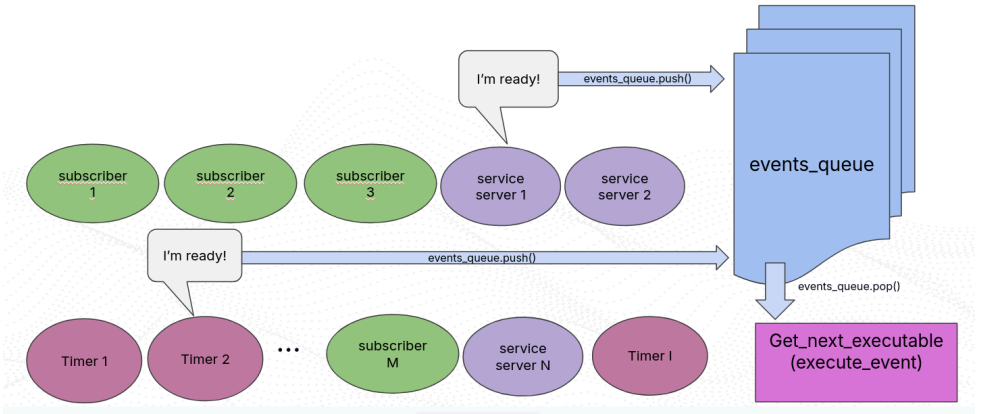
```
{
  std::lock_guard wait_lock{wait_mutex_};
  // every worker thread serializes here!
  if (!get_next_executable(any_exec, timeout)) continue;
}
execute_any_executable(any_exec);   // only this runs in parallel
```



The discovery half of the loop is still serial across all threads.

`EventsExecutor`

- Entities register an `rmw_event_callback_t` **at construction time**.
- The **middleware** signals readiness — message arrival, request, response, QoS event.
- Events are pushed onto a thread-safe **events queue**; the spin thread pops them **FIFO**.
- A dedicated **timer manager thread** enqueues timer events.



`EventsExecutor` — Proven, but Incomplete

> "efficiently run a navigation stack consisting of many nodes in a single process, with minimal CPU contention and a reasonable memory footprint, **on a Raspberry Pi 4**."

The gaps

1. No **callback group** concurrency — a single thread services the queue.
2. No support for **simulated time**.
3. **Fast-firing timers** whose callbacks exceed the timer period overflow the queue faster than the spin thread can drain it.

`EventsCBGExecutor`

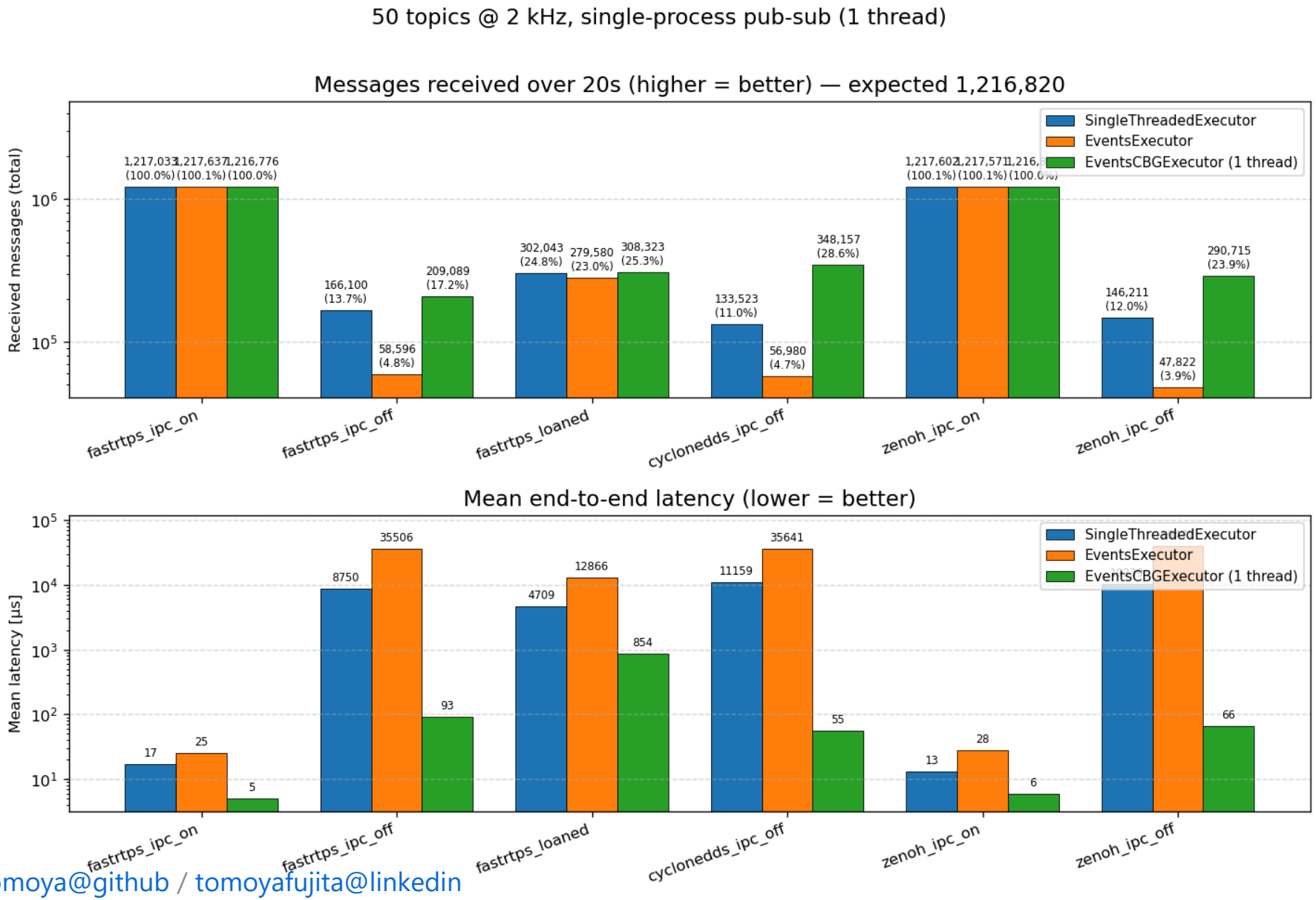
- Keeps the **events queue**; feeds it into a callback-group-aware `FIFOScheduler` serviceable by **multiple threads**.
- Timer manager restructured: multiple **ROS time sources**, and an actively firing timer can no longer be serviced twice.
- The globally serialized "find work" step is **gone** — workers pull from the scheduler independently.

```
sync_callback_groups();
auto ready = scheduler->get_next_ready_entity();   // no global lock
if (ready.moreEntitiesReady) scheduler->unblock_one_worker_thread();
ready.entity->execute_function();
```

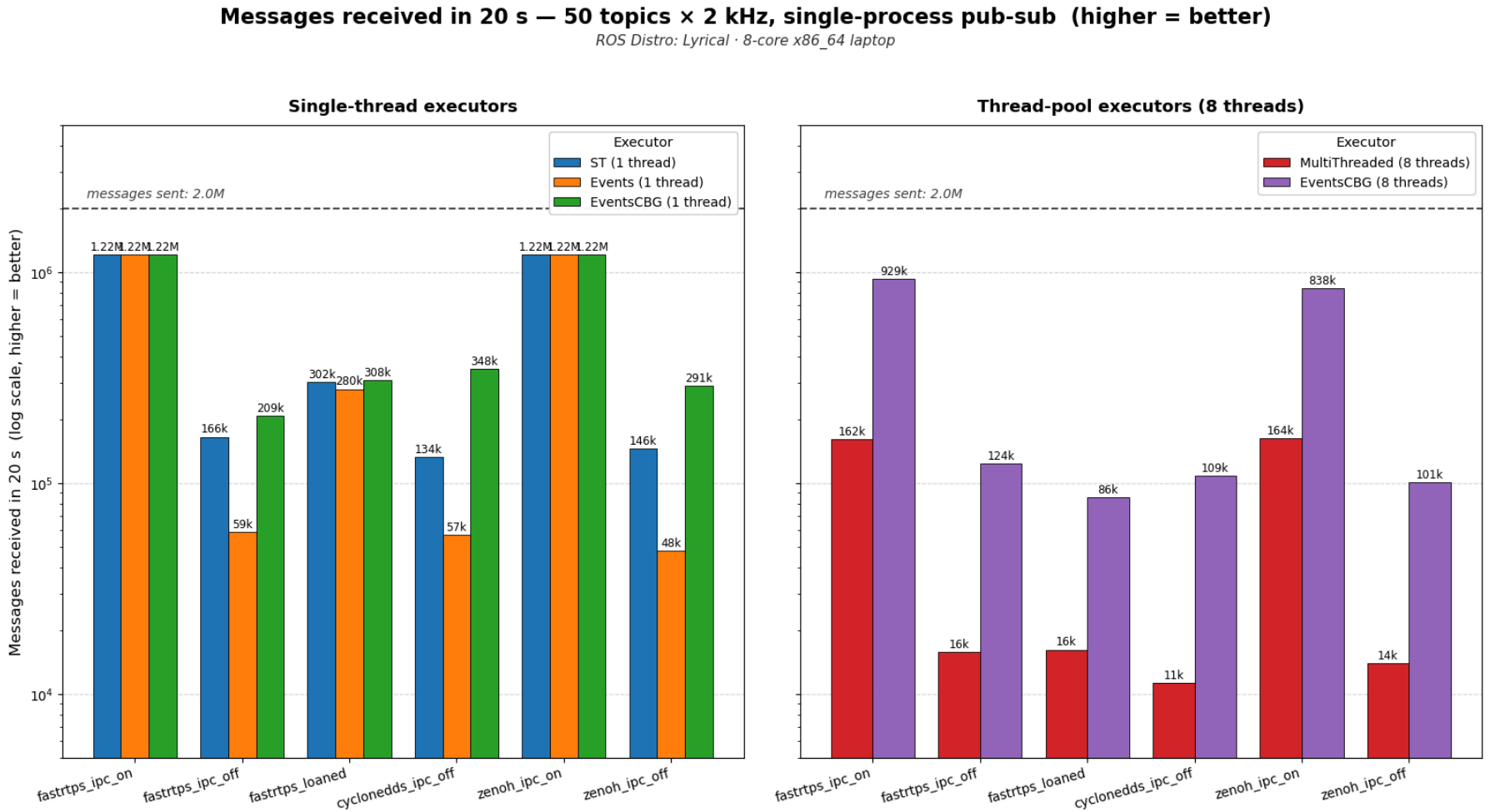
Benchmark

- Tool: `ros2-benchmark-container` on Lyrical testing images.
- Executors compared: `SingleThreaded`, `MultiThreaded`, `Events`, `EventsCBG` (1 and 8 threads).
- Topology: **50 topics, 63–2000 Hz, single process** → **~60,000 msg/s** for 20 s ≈ **1.2 M payloads @ 1 MB**.
- Transports: **intra-process (`ipc_on`)** and **through the middleware** (Fast DDS, Cyclone DDS).
- QoS: `rclcpp` default — Reliable, Keep Last, Volatile, depth 10.
- Hardware: 8-core x86 Lenovo developer laptop; ~4 h per full suite.

Metrics: **throughput**, **mean end-to-end latency**, **CPU %**.


50 topics @ 2 kHz, single-process pub-sub (1 thread)
Messages received over 20s (higher = better) — expected 1,216,820
Received messages (total)
SingleThreadedExecutor
EventsExecutor
EventsCBGExecutor (1 thread)
1,217,033 (100.0%)
1,217,637 (100.1%)
1,216,776 (100.0%)
166,100 (13.7%)
58,596 (4.8%)
209,089 (17.2%)
302,043 (24.8%)
279,580 (23.0%)
308,323 (25.3%)
133,523 (11.0%)
56,980 (4.7%)
348,157 (28.6%)
1,217,602 (100.1%)
1,217,571 (100.1%)
146,211 (12.0%)
47,822 (3.9%)
290,715 (23.9%)
fastrtps_ipc_on
fastrtps_ipc_off
fastrtps_loaned
cyclonedds_ipc_off
zenoh_ipc_on
zenoh_ipc_off
Mean end-to-end latency (lower = better)
Mean latency [µs]
17
25
5
8750
35506
93
4709
12866
854
11159
35641
55
13
28
6
Average process CPU

- Only **intra-process** setups comfortably reach 1.2 M messages (~**100%** of expected).
- `EventsCBGExecutor` in **1-thread mode** services **as many or more** callbacks than the others, **at significantly lower latencies**, across every transport.
- The **denser** the workload, the bigger the gap.


Messages received in 20 s — 50 topics × 2 kHz, single-process pub-sub (higher = better)
ROS Distro: Lyrical · 8-core x86_64 laptop
Single-thread executors
Thread-pool executors (8 threads)
Messages received in 20 s (log scale, higher = better)
messages sent: 2.0M
Executor
ST (1 thread)
Events (1 thread)
EventsCBG (1 thread)
MultiThreaded (8 threads)
EventsCBG (8 threads)
1.22M
166k
59k
209k
302k
280k
308k
134k
57k
348k
146k
48k
291k
162k
929k
16k
124k
86k
11k
109k
164k
838k
14k
101k
fastrtps_ipc_on
fastrtps_ipc_off
fastrtps_loaned
cyclonedds_ipc_off
zenoh_ipc_on
zenoh_ipc_off

Using all available cores:

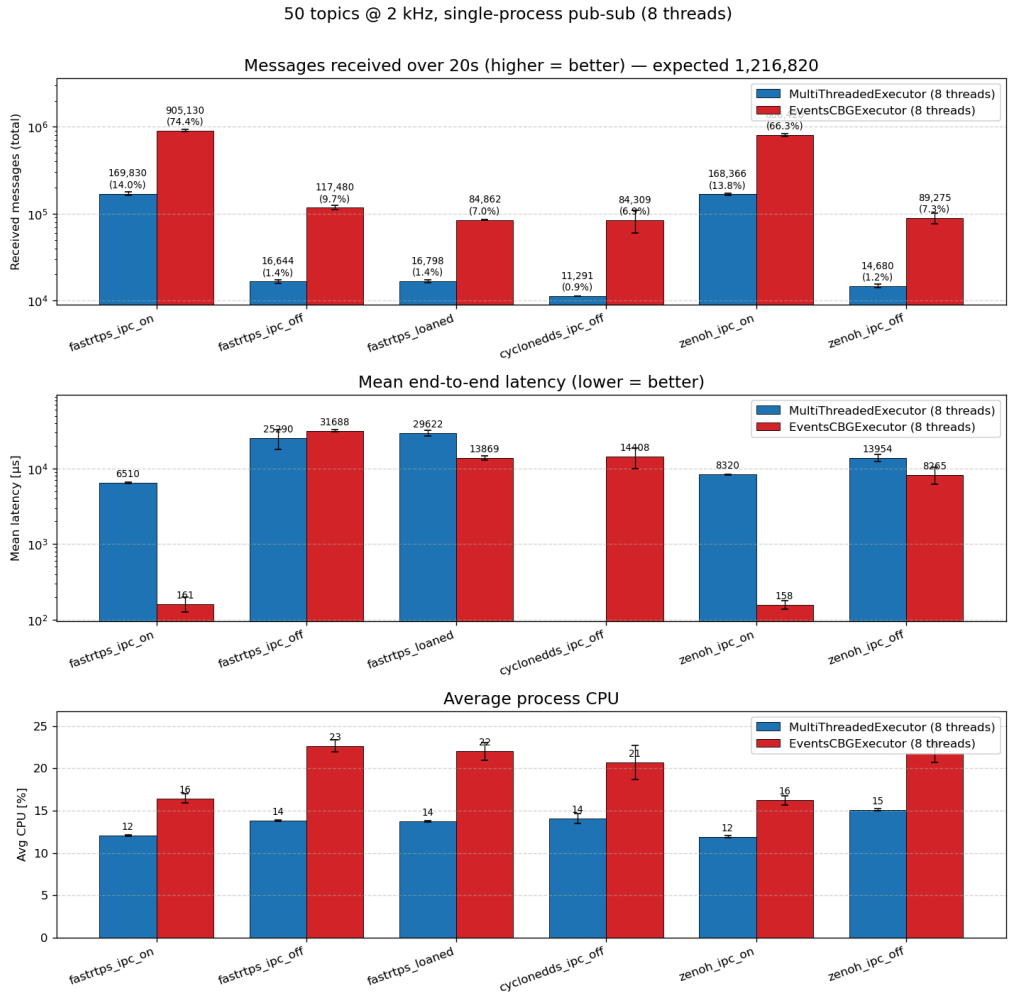
- Best multi-threaded result: **CBG + intra-process ≈ 75%** throughput — still *below* single-threaded.
- Through the middleware: **~10%** (CBG) vs. **<2%** (classic `MultiThreadedExecutor`).
- With Cyclone DDS, classic MT overhead was so severe that **4 of 50 subscribers received zero messages** in 20 s → **NaN** latency.

CPU — Cost vs. Value

> "The callback group executor is spending about **1.3 to 1.75x more CPU** than the old polling-based multi-threaded one, but look at what it's getting for that extra spend — in the intraprocess cases, **up to 6x more throughput and 50x less latency!**"

The old executor's *lower* CPU number is not efficiency:

> "a symptom of **core starvation** because the scheduling is too slow."



Caveats — None of This is Magic

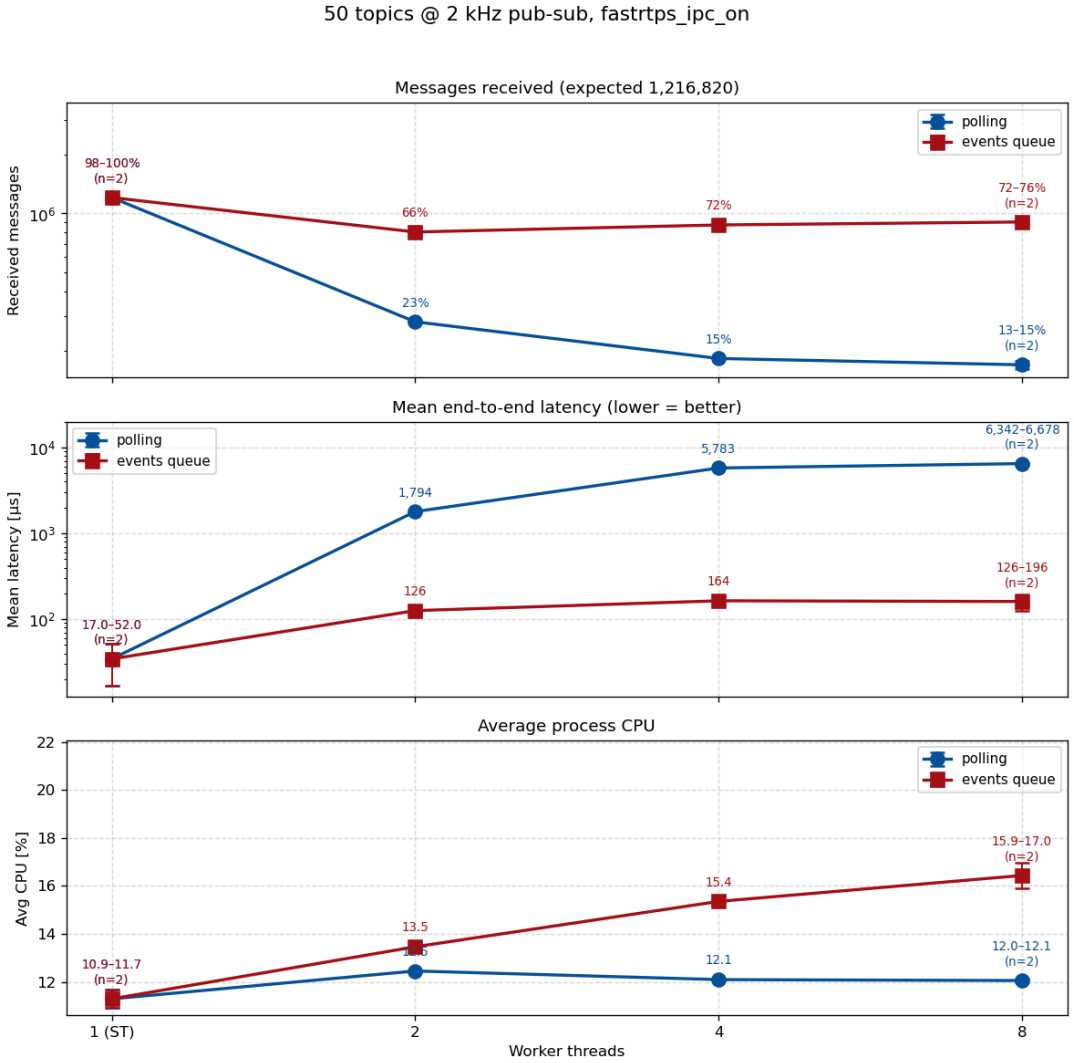
- Gains scale with **entity count**. Few entities → little to win.
- The executor only removes **executor overhead**:

> "None of the efficiency gains ... will help much if your **callbacks are the bottleneck**, or reduce the inherent overhead of your messages going through the **rmw layer**."

- **Unbounded queue risk**: if the CPU is overloaded, the events queue can grow without bound as it fails to keep up with the backlog — degrading latency for *everything* else.

How Many Threads Need?

- Default to **single-threaded** for raw throughput.
- Reach for threads only for genuinely CPU-heavy callbacks — planning, machine vision, sensor processing.
- **Measure** the requirement; don't just pass `std::thread::hardware_concurrency()`.



Takeaways & References

- **Lyrical / Rolling**: `EventsCBGExecutor` available in `rclcpp` today.
- **Jazzy**: backported (June 2026 sync).
- Prefer **events over polling**; prefer **fewer threads**; declare **callback groups** deliberately; then **measure**.

References

- The Evolution of Execution Management in rclcpp — Skye Medeiros, Polymath Robotics
- About Executors — ROS 2 Concepts
- ros2/rclcpp · Client Library Working Group